Plant-Agnostic Disease Classification via Domain-Adversarial Transfer Learning

Anna Titizyan

April 19, 2026

Abstract

This project studies plant-agnostic plant disease recognition from leaf images. The dataset is organised in folders whose names combine a host plant and a disease, but the prediction target is the disease label only. This creates a shortcut-learning risk: a model can achieve strong validation accuracy by relying on plant-specific cues instead of disease morphology, and then fail when the host distribution changes.

To address this, I train a domain-adversarial model where the host plant acts as the domain label during training. Using a Gradient Reversal Layer (GRL), the feature extractor is encouraged to be predictive for disease while becoming uninformative about plant identity. I evaluate both a standard in-distribution split and a held-out-plant protocol that explicitly tests host invariance.

Introduction

Automated plant disease detection can support earlier intervention in crop management and reduce unnecessary pesticide use.

What I here discuss is classifying the disease alone, regardless of which plant it appears on. Formally, given input image x, predict label $y_d \in \mathcal{D}$ (disease) while being invariant to domain $y_p \in \mathcal{P}$ (plant species). This framing naturally maps to domain adaptation: the plant species is the domain, and I want representations that are discriminative for disease but uninformative about plant identity.

To achieve this, I use the Gradient Reversal Layer (GRL) of Ganin et al. [1], which produces domain-invariant features by adversarially training the feature extractor to fool a domain classifier. My contributions are:

- A domain-adversarial training pipeline on a real-world plant disease dataset.
- Ablations over backbone freeze depth, GRL scheduling, domain loss weight, and class-balancing strategy.
- A two-protocol evaluation: random split (in-distribution) and held-out-plant split (out-of-distribution) to measure true plant-agnosticism.

Dataset

Structure

The dataset contains approximately 8,000 images split into train and val directories. Each subdirectory is named with the convention plant disease (e.g., apple black rot, tomato late blight). I parse directory names by splitting on whitespace: the first token is the plant species, and the remaining tokens form the disease label.

From this parsing I extract:

- **Unique diseases:** 40 classes (e.g., black rot, early blight, late blight, powdery mildew).
- **Unique plant species:** 38 hosts.
- The dataset is naturally imbalanced—some disease classes have many more images than others.

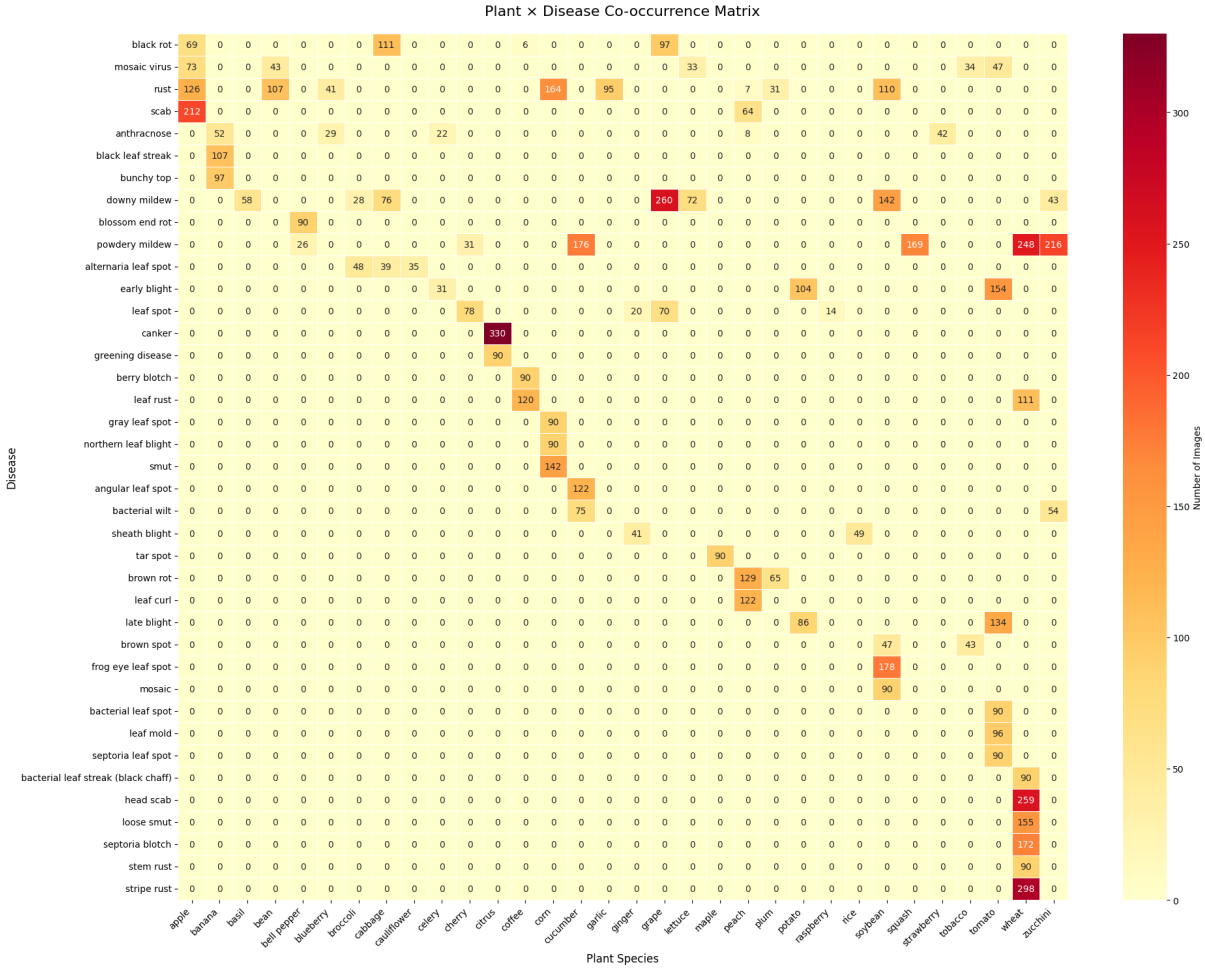

Figure 1: Plant–disease co-occurrence counts in the training set. The matrix is sparse: many disease labels appear with only one or a few host plants, while a small subset (e.g., rust and powdery mildew) appears across multiple hosts. This sparsity is important for interpretation: if a disease occurs only on a single host in the data, host-invariant learning is fundamentally underdetermined.

Label Extraction

The key insight motivating the label extraction is that the same disease word (e.g., "late blight") must be treated as a single class across all host plants. One complication arose during parsing: one plant species in the dataset has a two-word name — bell pepper. A naive whitespace split would assign plant="bell" and disease="pepper [disease]", creating spurious disease classes. To handle this, the parser checks for known multi-word plant names before falling back to the first-token heuristic:

Listing 1: Directory parsing with multi-word plant name handling.

```
MULTI_WORD_PLANTS = {"bell pepper"}

for folder in folders:
    name = folder.name.lower().strip()
```

```
    if name == ".ds_store":
        continue

    matched_plant = next(
        (p for p in MULTI_WORD_PLANTS if name.startswith(p + "␣")), None
    )
    if matched_plant:
        disease_name = name[len(matched_plant)+1:]
    else:
        parts = name.split()
        matched_plant = parts[0]
        disease_name = "␣".join(parts[1:])

    disease_set.add(disease_name)
    plant_set.add(matched_plant)
```

This yields two integer-indexed dictionaries: `disease2idx` (the target label) and `plant2idx` (the domain label used only during training).

Validation Protocols

I use three Different approaches:

Protocol A — Random Split. The official train/val directory split is used directly. The validation set shares the same plant species distribution as the training set. This measures *in-distribution* performance.

Protocol B — Held-Out-Plant Split. I merge all data and hold out entire plant species for testing (bean, tobacco, broccoli),while ensuring that the remaining species in the training set contain overlapping disease categories. This directly measures whether the model has learned plant-invariant disease representations, since the test images come from species never seen during training.

Protocol C — No GRL and Held-Out-Plant Split. Here I again merge all data and hold out one plant (cabbage) with many images and that has several diseases. The Model is without GRL and has one classification head. This helps understanding whether we need GRL or not.

Methodology

Architecture Overview

The full model has three components:

1. **Backbone**: EfficientNet-B0 pretrained on ImageNet-1k (via `timm`), used as a feature extractor with `num_classes=0` to expose raw features of dimension $d = 1280$.
2. **Projector**: a single-layer that maps $\mathbb{R}^{1280} \to \mathbb{R}^{512}$ with BatchNorm, ReLU, and Dropout(0.3). This shared representation is the input to both heads.
3. **Disease head**: linear $512 \to 256 \to |\mathcal{D}|$ with ReLU and Dropout(0.3), trained to minimise disease cross-entropy.
4. **Domain head**: identical in structure $512 \to 256 \to |\mathcal{P}|$, trained to classify plant species—but receives gradients through a *Gradient Reversal Layer* (GRL), which flips the sign of gradients during backpropagation. This forces the projector to produce features that are *uninformative* about plant identity.

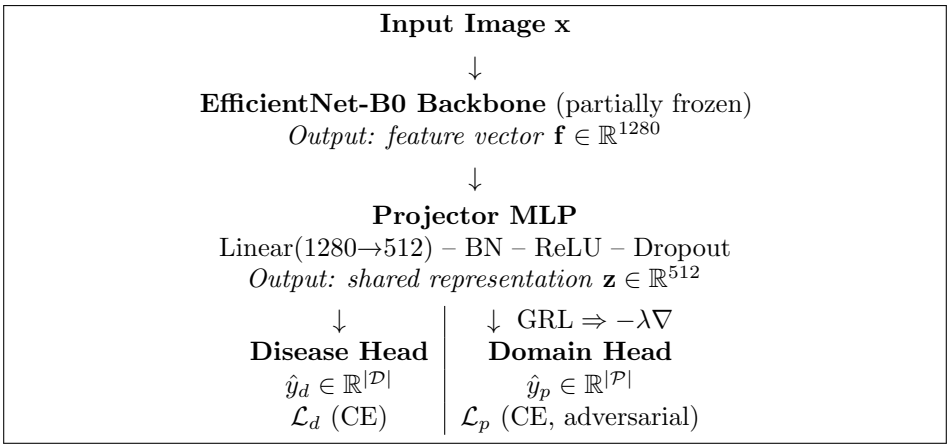

Figure 2: Model architecture.

Gradient Reversal Layer

The Gradient Reversal Layer (GRL) is an identity mapping in the forward pass. During backpropagation, it multiplies the gradient by $-\lambda$. In practice, this means that while the domain head is trained to minimise domain cross-entropy, the feature extractor receives the negated domain gradient and is therefore trained to maximise the domain loss (i.e., to make plant identity hard to predict from features).

Let $z = G_z(x)$ be the shared representation and. $\mathcal{L}_p$ is the domain classification loss.Then the gradient reversal layer modifies the gradient as:

$$\left.\frac{\partial \mathcal{L}}{\partial z}\right|_{\text{GRL}} = -\lambda \frac{\partial \mathcal{L}_p}{\partial z}$$

Listing 2: GRL forward and backward passes.

```python
class GradientReversalFn(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, lambda_):
        ctx.save_for_backward(torch.tensor(lambda_))
        return x.clone()

    @staticmethod
    def backward(ctx, grad_output):
        lambda_, = ctx.saved_tensors
        return -lambda_ * grad_output, None
```

GRL Scheduling

In early epochs, the domain classifier is essentially random and can produce high-variance gradients. If those gradients are immediately reversed into the shared representation, the feature extractor can be pushed in unhelpful directions before the disease signal is established.I initially tried from epoch 1 and saw unstable validation mAP oscillations. Therefore I use a short warm-up where λ= 0 for the first 4 epochs, and then increase λ smoothly with a sigmoid ramp.

- $\lambda = 0$ for the first 4 epochs (disease head trains freely).
- After epoch 4, λ follows:

$$\lambda(e) = \frac{2}{1 + e^{-5 \cdot e/E}} - 1$$

where e is the current epoch and E the total number of epochs. This sigmoid ramp starts near 0 and saturates at 1, ensuring adversarial pressure builds gradually.

Training Objective

The total loss is:

$$\mathcal{L} = \mathcal{L}_d + \alpha \cdot \mathcal{L}_p$$

where $\mathcal{L}_d = \mathrm{CE}(\hat{y}_d, y_d)$ is disease cross-entropy, $\mathcal{L}_p = \mathrm{CE}(\hat{y}_p, y_p)$ is domain cross-entropy (reversed at the projector), and $\alpha = 0.3$ is a hyperparameter weighting the domain term. At inference time, only the disease head is used and the GRL coefficient is set to $\lambda = 0$.

Backbone Freezing Strategy

EfficientNet-B0 is initialised from ImageNet weights. All parameters are frozen except the last two convolutional blocks (`blocks[-2:]`), the projector, and both classification heads.In early experiments, unfreezing only the final convolutional block did not provide sufficient representational flexibility, the model tended to rely too heavily on the fixed ImageNet features, which were not fully aligned with the fine-grained plant disease domain.

Unfreezing an additional block improved optimization dynamics particularly in disease classification accuracy and mAP. This suggests that higher-level convolutional features still require task-specific adaptation, while lower layers remain largely general-purpose.

Data Augmentation

Training transforms:

- RandomResizedCrop(224, scale=(0.7, 1.0)) — simulates partial views.
- RandomHorizontalFlip(p=0.5)
- RandomRotation(degrees = 10)
- ColorJitter(brightness=0.3, contrast=0.3, saturation=0.25, hue=0.05) — plant images show significant lighting variation.
- RandomErasing(p=0.25, scale=(0.02, 0.15)) — regularisation.
- ImageNet normalisation.

Validation transforms: Resize to 256, CenterCrop to 224, ImageNet normalisation (deterministic).

Optimisation

- **Optimiser:** AdamW, 10^{-4}, weight decay 10^{-4}.
- **Scheduler:** CosineAnnealingLR, $T_{\max} = 30$, $\eta_{\min} = 10^{-6}$.
- **Batch size:** 32.
- **Epochs:**30.
- **Hardware:** Apple MPS (M-series) locally; Google Colab T4 GPU.

Evaluation Metric

I report two metrics:

- **Top-1 Accuracy** — fraction of images for which the argmax prediction equals the ground-truth disease.
- **mAP (mean Average Precision)** — macro-average AP across all disease classes, computed from predicted softmax probabilities. mAP is the primary metric because it accounts for confidence calibration and is robust to class imbalance.

Ablation Study

Experiments are tracked in W&B. The key design choices I varied across runs are: backbone freeze depth, GRL scheduling, domain loss weight α, and class-balancing strategy. Results for each run are reported directly from W&B logs.

Effect of the Gradient Reversal Layer

The most direct ablation is removing the GRL entirely, keeping everything else identical — same backbone, same projector, same disease head, same training configuration. Without the GRL the model has no adversarial pressure to forget plant identity and is free to exploit host-specific visual cues as a shortcut.

Table 1: With GRL vs. baseline without GRL on the validation set.

Model	**Top-1 Acc (%)**	**mAP**
Protacol C (Without GRL + held-out plant)	6.96	0.4048
Protacol B	44.84	0.6349

Removing the GRL causes a large drop in disease-label performance. One plausible explanation is shortcut learning: when plant species is correlated with disease labels, the network can allocate capacity to plant-identifying cues that do not transfer when the evaluation target is "disease only".

Effect of Backbone Unfreezing

Unfreezing the last two convolutional blocks (`blocks[-2:]`) rather than just one gave a consistent improvement in validation mAP. Unfreezing the full backbone led to overfitting given the dataset size (≈8k images), so I settled on two blocks.

GRL Scheduling

Applying the GRL from epoch 1 destabilised early training — the domain head is randomly initialised and its gradients are noisy, which corrupts the shared representation before it has learned anything useful. Adding the 4-epoch warmup ($\lambda = 0$ initially, then the sigmoid ramp) fixed this and improved final mAP.

Domain Loss Weight α

I ran experiments with $\alpha \in \{0.1, 0.3, 0.5\}$. $\alpha = 0.3$ gave the best validation mAP. Too small and the domain signal is too weak to push the projector toward plant-invariant features; too large and it over-penalises the projector at the expense of disease discriminability.

Class Balancing

The disease distribution is imbalanced. I use a WeightedRandomSampler where each sample is assigned weight proportional to $1/\sqrt{n_c}$ (square-root frequency smoothing). This is less aggressive

than $1/n_c$ but still oversamples rare classes, preventing the model from collapsing to majority-class predictions.

Results

Primary Results

The best model achieves the following on the official validation set:

Table 2: Final model performance.

Split	Top-1 Acc (%)	mAP
Protocol A (random train/val)	70.00	0.7818
Protocol B (held-out: bean, tobacco, broccoli)	44.84	0.6349

The ∼25 pp accuracy gap between Protocol A and Protocol B is expected: Protocol B tests generalisation to plant species entirely unseen during training. The fact that mAP remains at 0.63 rather than collapsing (compare to 0.40 without GRL on Protocol A) suggests the adversarial training does transfer some disease-discriminative structure across hosts.

Per-Class Observations

From qualitative inspection of top-5 predictions, the model shows high confidence on visually distinctive diseases (e.g., apple scab, powdery mildew) and lower confidence on diseases with subtle early-stage symptoms (e.g., early blight vs. late blight in the same host). The GRL does appear to help: disease predictions are more consistent when the same disease appears on different hosts, compared to the baseline without GRL.

Experiment Tracking

Almost all experiments are tracked in Weights & Biases under the project Plant_Disease_Classification. The public run URL is included in the repository README. Each run logs train and validation mAP per epoch, learning rate, and GRL λ value.

Production API

The model is served via a FastAPI endpoint that accepts a single image file and returns the top-5 disease predictions with confidence scores. The API runs locally, with model weights loaded from `withtwon/plant-disease-model` on HuggingFace.

Listing 3: FastAPI endpoint.

```python
from fastapi import FastAPI, UploadFile, File
import torch
from PIL import Image
import io
from inference import load_model, VAL_TF, device

app = FastAPI()
model, idx2disease = load_model()

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    image = Image.open(io.BytesIO(await file.read())).convert("RGB")
```

```
    image = VAL_TF(image).unsqueeze(0).to(device)

    with torch.no_grad():
        logits, _ = model(image, lambda_=0.0)
        probs = torch.softmax(logits, dim=1)[0]
        top5 = probs.topk(5)

    results = [
        {"class": idx2disease[i.item()], "prob": float(p.item())}
        for p, i in zip(top5.values, top5.indices)
    ]
    return {"predictions": results}
```

No third-party inference APIs are used; the model runs entirely locally.

Discussion

Why the GRL Helps (and Its Limits)

The adversarial training approach provides a principled way to encourage the feature extractor to be plant-invariant. The results show consistent improvement over the no-GRL baseline on Protocol A. However, the held-out-plant protocol reveals that the learned invariance is incomplete. Two fundamental constraints limit the approach here:

1. **Label coverage asymmetry.** Some disease classes in the dataset only occur in one plant species (e.g., a disease unique to apple). No amount of adversarial training can separate disease from plant when there is a one-to-one mapping between them in the data.
2. **Visual co-occurrence.** Certain disease phenotypes manifest very differently on different hosts, making a plant-agnostic representation inherently harder—the disease "late blight" looks different on tomato and potato at the pixel level.

Limitations

- The dataset is relatively small (8k images) compared to large-scale benchmarks, which limits deep fine-tuning and held-out generalisation.
- The GRL approach assumes that plant species labels are available at training time; in a fully unlabelled setting, a different domain adaptation strategy would be needed.

Conclusion

I present a domain-adversarial approach to plant-agnostic disease classification. By augmenting an EfficientNet-B0 backbone with a gradient reversal layer and a plant-domain adversarial head, the model learns disease representations that are progressively decoupled from host identity. Ablations identify GRL scheduling and domain loss weight as the two most critical hyperparameters. The held-out-plant evaluation protocol directly tests whether the model generalises to unseen species, exposing remaining plant-specificity in the learned features and motivating future work on stronger invariance constraints and larger, more diverse datasets.

All code, model weights, and experiment logs are publicly available at github.com/titizyananna/plant-disease. The FastAPI endpoint is live at the URL provided in the repository README.

References

[1] Y. Ganin, E. Ustunova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. Lempitsky. *Domain-adversarial training of neural networks.* Journal of Machine Learning Research, 17(59):1–35, 2016.

[2] M. Tan and Q. V. Le. *EfficientNet: Rethinking model scaling for convolutional neural networks.* Proceedings of the 36th International Conference on Machine Learning (ICML), 2019.